findsprop AI chat assistant: architecture (ADR-128, proposed)

Customers only. The model can only call typed tools; every real write is a human click on an existing action.

Customer's browser

/ai-search page
Server Component shell (flag-gated)

Chat island (client)
Transcript kept in sessionStorage
Sends last ~12 turns + page context
Shows streaming, limit states, errors
Example prompts, no-personal-details note

Result cards
Existing property and project cards
Drawn from ids the tools returned,
never from links the model writes

Sign-in gate
After 2 free anonymous messages
Existing phone sign-in, then back to
/ai-search with the transcript kept

Confirm card
Shows the real home or project, the customer's name and verified phone, and the message. Customer clicks Send.

request

stream

click Send

Vercel: the Next.js 16 app

POST /api/ai/chat (streaming Route Handler)

red = guardrail

1 Validate + input pre-check | 2 Session (optional) | 3 Rate limits
4 Agent loop | 5 Output check + ids | 6 Usage row (no text)

Agent loop (server side only)
Max 5 rounds, 6 tool calls in all
Reads in parallel (max 3 a round)
Last round: answer only, no tools
Own code, no agent framework

AI provider adapter (server/ai)
One neutral interface
OpenAI now, Gemini later
Fake provider for tests and CI
Timeouts, caps, error mapping

tool calls

Tools: the model's only way to act

Read tools
search_properties
get_property
compare_properties
search_projects
get_project
get_builder
list_localities

Maths
calculate_emi
(pure function)

Privacy rule
No tool takes a
user id, phone or email

Propose only
propose_enquiry
propose_site_visit

Write nothing. They return card data.

card data

Existing cached public reads (publicAnon, explicit columns, RLS)
getSearchResults · getProperty · getComparedProperties · getProjects · getProject

Existing Server Actions (defineAction)
sendEnquiry · submitProjectLead
Verified phone, rate limit, idempotency key, then the database function
Reused as they are: no new write path

3 count

model

6 usage

reads

RPC

Model provider and Supabase

Rate-limit counters
Trial: 2 msgs per cookie id (or hash)
Anonymous cap per IP /64 per day
Burst: 10 per minute
Signed-in cap per user id per day
Global daily budget

Model provider API
OpenAI first, Gemini later (swap by env)
Receives messages and tool results
Stateless calls, nothing kept by us

ai_usage_events (proposed)
Time, who, model, tokens, tools, outcome
Never message text, phone or email

Controls and operations
FLAG_AI_SEARCH off = 404 everywhere
Logger redacts; no message text logged
Cost and abuse reporting from usage rows

Postgres with RLS
Approved homes, live projects, units,
price trends, builders (public columns only)

enquiries and project_leads
Written only by submit_enquiry and submit_project_lead (server-only RPCs) plus the audit trail, as today

Facts come from tools
Prices, availability, RERA and EMI come from tool output. Money is formatted by code.

Links are built by code
The model names ids. The server checks each id came from a tool this turn.

Writes need a human click
The model proposes. The customer presses Send. Existing actions do the write.

Hostile text stays data
Listing and builder text is marked untrusted. No tool writes or fetches.

Other people stay private
No tool takes a user id, phone or email. Phones are masked before the model sees them.

Colours: blue = browser, grey = app, purple = AI, green = data, orange = anything that can write for the customer. Red = guardrail. Dashed = proposed or optional.